Controlling the body through support forces

M20 • 07 Jul 2028 - 31 Jul 2028 • 75 hours

Final outcome

Implement stance-leg force allocation as a quadratic programming problem, QP. From the desired total body force and moment, current contact points and constraints, it determines admissible support forces and a diagnostic status. Compare the result in simulation with the verified classical mode from M19. Test slopes, small disturbances and solver failure. Prepare qp_stance_v1 with a working fallback mode and solve-time measurements.

Determine when requested support forces are physically feasible. The achievable body moment depends on leg placement: contact cannot pull on the ground, friction limits lateral force and actuators have limits. If a command is unreachable, the controller must report it and select a verified mode. A small optimisation residual is meaningful only when the model, constraints and result timing are correct.

Prerequisites

From M19, bring the slow gait, classical standing and pause modes, orientation and velocity estimates, foot coordinates, reliable supports and contact-loss log. Earlier modules supply masses, centres of mass, kinematics, the Jacobian, joint limits and actuator model. Check units and parameter sources. Label illustrative simulation currents and friction as educational values; they do not replace measured hardware characteristics.

Start with four unchanged contacts and a stationary body, then move to three supports and contact switching. This separates force-allocation errors from step-planning errors. If the gait is unstable, keep the legs in stand mode, retain the floating body and verify standing before connecting QP to swing. Fixing the body in the world does not demonstrate force-control quality.

Whole-body control and MPC

Study the centre of mass, momentum, desired force-and-moment vector (wrench), friction pyramids, conditioning, incompatible constraints, warm starting from the previous solution, mapping through the transposed Jacobian, impedance and task priorities. Study the structure of a complete inverse-dynamics controller. For MPC, conduct a small finite-horizon experiment; for NMPC, examine the problem formulation.

This month's result is classical support control under specified simulation conditions. Running, jumping and push tests on a real robot are outside its scope. Preserve the verified mode unchanged for comparison in M21: a learned policy must build on completed checks of the baseline force-control loop.

75 hours: from the model to verified failure handling

Work plan

Hours 1-20, theory 10 / practice 10: dynamics about the centre of mass, momentum, forces and moments, contact changes, units and balance. Build the matrix linking foot forces to the body's force-and-moment vector, and analytical check cases. Compare M19 quasi-statics with a centre-of-mass acceleration model; list effects still excluded.

Hours 21-40, theory 10 / practice 10: QP formulation, friction and normal-force constraints, conditioning, regularisation, incompatible constraints and reuse of the previous solution. Prepare a solver with independent output checks, status logging and timing measurements. First make a small problem correct: increasing the horizon does not eliminate sign errors.

Hours 41-60, theory 10 / practice 10: converting contact forces into joint torques, impedance, priorities, torque and current limits, saturation and inverse dynamics. Connect the actuator model and check limits on the complete command. The sum of force and position controllers can exceed the limit even when each is individually admissible.

Hours 61-75, theory 5 / practice 10: finite-horizon MPC, the idea of NMPC, repeatable slopes and disturbances, and return to verified standing. Compare the baseline with QP, test solver failures and describe operating limits. Total: 35 hours of study and analysis plus 40 of practice. Waiting for a solver does not count as a separate session.

Organising the exercises

The first three theory sections each comprise five 2-hour sessions. Practice comprises four Sunday blocks of 10 hours, with breaks excluded from counted time. The final 5 hours of analysis are 2 + 2 + 1. Numbered hours specify workload and order, not an additional calendar. Date boundaries may overlap the next module; rest, holidays and separate integration checks follow the curriculum, without duplicating hours.

Keep two configurations: the unchanged classical baseline and the current QP. For each experiment, record the change, expected effect, result and grounds for accepting the command. Include expected failures handled by the fallback mode in the failures log: they demonstrate protective behaviour even if the robot travels a shorter distance.

Do not spend time installing OSQP, Crocoddyl and acados simultaneously. The required work needs one QP solver and the familiar simulator; other tools illustrate possible next steps. Python suffices for formulation and analysis; add C++ only to an existing control loop if measurements demonstrate a need.

Centre of mass, momentum and force requests

Meaning of the model

Let c be the centre of mass in metres, m the total mass in kilograms and v the centre-of-mass velocity. Linear momentum is m·v, and its change is determined by external forces. For ground forces on the robot f_i, m·a = Σf_i + m·g, where g is the gravitational-acceleration vector. In a stationary stance, support force balances weight; a controller-requested acceleration requires the corresponding physical forces.

The moment of contact forces about the COM is Σ(r_i × f_i), where r_i = p_i − c is the foot's moment arm in metres. Force is measured in newtons and moment in N·m. This moment determines the change in angular momentum about the centre of mass. If the body is approximated as one rigid body, specify its inertia and assumptions. Leg motion also affects total momentum, especially at high accelerations.

Task 1. Check mass and centre of mass

Express every link's centre of mass in the common world frame and collect the link masses. Calculate c = Σm_j c_j / Σm_j. Compare a symmetric stance, a raised leg and a payload at a known point. Save the expected shift direction and calculated coordinates. Zero or negative mass and inconsistent units must cause explicit rejection by the model check.

Task 2. Assemble the force matrix

For n supports, assemble a force vector f of size 3n and a matrix A of size 6 × 3n: A·f = [Σf_i; Σ(r_i × f_i)]. The first three rows sum forces; the next three sum moments. Manually check a vertical force with a nonzero moment arm, then two equal and opposite forces producing a nonzero moment. Express r and f in the same coordinate frame.

Task 3. From pose error to a request

Use COM position and velocity errors to request a bounded acceleration a_des. The required contact force is m(a_des − g). Obtain the desired moment from a small orientation error and angular velocity, stating the frame and approximation. Limit the rate of change of requests before optimisation. At zero error, check weight compensation; at a small positive error, check the direction of the restoring action.

Compare quasi-statics, a_des=0, with a small nonzero acceleration. Show the force changes and explain why achievable moments differ between three and four supports. Increase controller gains gradually for your own simulation; these values are not universal settings for all actuators.

QP: admissible forces take priority over an ideal request

Problem formulation

Minimise the weighted error $||A \cdot f - w_{des}||^2_Q$, adding $\lambda||f||^2$ and, if needed, $\gamma||f - f_{prev}||^2$. Here w_{des} is the desired total force and moment, Q contains the selected weights, and λ and γ are nonnegative regularisation coefficients. The last term smooths solution changes but must not retain force at a lost contact. Reconcile force and moment scales through normalisation or a characteristic length; otherwise, weights depend implicitly on units.

Task 4. Contact constraints

For each foot, define local tangential axes and a surface normal. Normal force f_n is nonnegative and bounded above by the scenario limit. During swing, every contact-force component is zero. Take the friction coefficient μ from the surface model and investigate its uncertainty. Negative μ and an invalid normal must be rejected by input validation.

For a simple conservative pyramid, use $|f_{t1}| + |f_{t2}| \le \mu f_n$, represented by four linear inequalities $\pm f_{t1} \pm f_{t2} \le \mu f_n$. This diamond-shaped region lies inside the circular Coulomb cone. The two independent constraints $|f_{t1}| \le \mu f_n$ and $|f_{t2}| \le \mu f_n$ form a square whose corners extend outside the circular cone; do not call this approximation conservative without a correction.

Task 5. First solutions with hand-calculated answers

Solve the static symmetric problem on a horizontal plane. With symmetric geometry and equal weights, expect equal vertical loads summing to $m \cdot g$ and near-zero lateral forces. Disable one support, update A and constraints, and compare the distribution. Check Af, normal forces and limits yourself, independently of solver status.

Request a deliberately excessive lateral force. With a soft penalty on desired force-and-moment deviation, the problem may remain feasible while tracking error grows. Incompatible hard equalities produce infeasible status. Distinguish these cases: solved does not mean that the body's request is met exactly. Set an acceptable residual for your scenario.

Save qp_cases.json with initial conditions, solutions, independent constraint checks and explanations of physical meaning. State the coordinate frame for every force. The ground reaction on the leg and the leg's force on the ground are opposite.

Numerical stability and solver failures

What to study about numerical computation

Poor conditioning arises from disparate variable scales or geometry that can barely produce force or moment in a particular direction. Regularisation smooths a solution but does not add physical support. Compare widely spaced and nearly coincident contacts. Identify which moments are difficult to produce; a condition number alone is insufficient explanation.

Task 6. Scaling and the initial guess

Solve an example in consistent SI units, then deliberately supply a moment arm in millimetres instead of metres. Unit checking must detect the data-format error. If the solver still outputs a number, show the change in physical moment. On correct data, compare normalised and unnormalised problems. Record residual and iteration count alongside the final forces.

For nearby successive states, compare starting from zero with starting from the previous solution. When contact changes, zero or transform invalid components of the old force: a swinging leg must not retain load. A fixed vector of 12 components can be retained, with inactive contacts set to zero by constraints. If dimensionality changes, explicitly update the leg-index mapping.

Task 7. Monitor status and deadlines

Handle solved, inaccurate, infeasible, maximum iterations, time limit and internal errors according to the pinned OSQP version's API. Define accepted statuses and tolerances in the configuration. Independently check every solution for finite values, inequalities, tracking error, current contacts and result age. A support may disappear during computation, making even a feasible but late result unusable.

Test an iteration limit, artificial delay, contradictory constraints and NaN input. Save reject events with precise reasons and the transition to classical mode. Computation in the critical loop requires external timing supervision: a hung call must not prevent the observer from detecting a missed deadline and prohibiting a new step.

Measure solve time

On the CPU, measure setup, update and solve separately. After warm-up, save a series of solutions with a specified range of contact configurations; report observation count, p50/p95/p99, maximum and deadline misses. Repeat under moderate background load and at cold start. Percentiles describe the measured sample but do not guarantee maximum time; account for this when selecting loop rate.

From foot force to actuator torque and current

Establish the sign conventions first

With the body fixed, the leg Jacobian relates $qdot_j$ to foot velocity. In the full model, J_c maps generalised velocity v to contact velocities; the environment's force on the robot contributes $J\_c^T f\_ext$. The dynamics are $M(q)\cdot vdot+h=S^T\tau+J\_c^T f\_ext$, where h includes gravity (p. 15). Therefore, $+J\_c^T f\_ext$ cannot be sent directly to motors: the equation determines its sign and the other terms. The leg's force on the ground is opposite to the ground reaction.

Task 8. Check virtual power

For a leg with the body fixed, choose a configuration, a small qdot and a known external force. Calculate $v\_F=J\cdot qdot$ and compare $f\cdot v\_F$ with $(J^T f)\cdot qdot$. With consistent frames and dimensions, the powers agree within numerical error. Then check the static torque of a single-revolute-joint model using the force moment arm. Save the sign and N·m unit checks before connecting the whole robot.

Connect QP through a mapping consistent with gravity compensation and the dynamics model. Show contact, gravitational and impedance contributions to the command separately. For an existing controller, first study its sign conventions; changing gain signs does not replace this check. The same rule applies to a finger: the object's force on the finger and the finger's force on the object are opposite.

Task 9. Limits on the complete command

At a fixed configuration and with the other terms known, express joint-torque limits as constraints on f. Determine them from the actuator, transmission, permissible current and load regime. In a simplified model, $\tau\_motor\approx k\_t\cdot I$; joint torque also depends on transmission ratio and efficiency. Do not assume that the controller's supply current equals winding current without verification. Peak torque at a stalled rotor is not the permissible continuous torque.

Before execution, check the sum of commands. Saturation after QP changes the resulting forces and moments; measure this change and include it in tracking error. Reduce the torque available to one leg and compare loads, body errors and transitions to fallback mode. Define simulation tolerances in advance. Record unknown real-actuator characteristics as required data for later transfer.

Impedance determines the response to pose and velocity errors, but stiffness is limited by the actuator and discretisation. Contact and actuator constraints must be satisfied first, then the body-holding task, then additional tasks. A weighted sum of penalties in one QP does not always enforce strict priorities. Study how this differs from hierarchical whole-body control.

The qp_stance_v1 laboratory

Interface and independent checking

Inputs: a fresh body estimate, COM and feet in a common coordinate frame, surface normals, contact set, desired forces and moments, and actuator limits. Outputs: forces, predicted torques, status, residual, solve time and acceptance or rejection reason. Retain the contact generation: a result for old supports cannot be accepted solely because its timestamp is fresh.

The testbed includes a request generator, QP construction, solver, independent validation and a switch between classical standing and pause modes. Start with M19 recordings without controlling the robot, then close the loop in simulation. Check constraints against the original physical data independently of solver diagnostics. This detects an incorrectly assembled matrix that the solver itself regards as a valid problem.

Task 10. Close the loop

Test, in sequence, four supports with weight compensation, a small COM shift, a small roll and pitch request, three supports and restoration of the fourth. Compare the desired force-and-moment vector, achievable Af and simulation response. State delays, actuator effects and model limits. Before motion, check signs and constraints in standing.

Compare the baseline and QP in identical scenarios. Measure RMS height and tilt errors, maximum deviation, fraction of time at a limit, solver-failure count, recovery time, falls and correct pauses. Reducing one error through continuous saturation does not constitute an overall improvement. Plot commands before and after every limit.

Task 11. Test fallback mode

If a solution is unavailable or rejected, prohibit new swings and use the M19 classical policy verified for the current supports. Do not replay old forces on a leg already in the air. The transition limits command jumps as far as current contacts allow; loss of support takes priority over smoothing. Holding may become impossible; then record scenario failure and stop further motion planning.

To return from fallback mode, require a stable, reliable estimate for a specified duration, several valid solutions and explicit state-machine permission. Oscillating solver status must not cause frequent switching. Save a diagram of rejection reasons, modes, contacts and commands. Demonstrate fallback operation for every planned artificial fault.

MPC and whole-body control

A finite horizon on a simple model

An ordinary support QP allocates forces for the current request. MPC predicts several future states and selects a command sequence subject to constraints. Execute the first step, then solve again using new measurements. Computational cost and accuracy depend on horizon, time step and model. With an incorrect model or data, increasing the horizon does not guarantee improvement.

Task 12. A conceptual MPC experiment

Use a one-dimensional COM model: x_next=x+v·dt, v_next=v+a·dt. The acceleration constraint represents available lateral force. Compare choosing only the current step with planning over a short horizon to stop before a boundary. Use identical initial x and v, boundary and constraints. Plot predicted and actual trajectories, then repeat with a mass error or command delay.

The experiment illustrates future constraints and replanning, but does not describe every quadruped contact. Record the assumptions: fixed contact, simplified dynamics, known boundary and no full rotational motion. The linear educational model is not NMPC. For a nonlinear formulation, list states, actions, contact sequence, constraints and required derivatives.

How whole-body control is structured

Study full inverse dynamics: floating-base state, mass matrix, Coriolis and gravitational terms, contact Jacobians, joint torques, and body and foot tasks. A floating base has no separate motors for each coordinate; contact forces produce body motion. Its desired pose must therefore be consistent with the legs and friction constraints.

Compare three levels: the quasi-static baseline distributes load during slow motion, QP accounts for current force constraints, and MPC adds prediction. State what the project implements and what is studied theoretically. Pinocchio, Crocoddyl and acados can help explain how these systems work; installing all three libraries is unnecessary.

Slope and disturbance

Specify a small slope with a known normal and an identical disturbance at one body point. Compare nominal μ with lower actual friction. Show when the model overestimates admissible forces and the solver accepts them while actual slipping occurs. Save the baseline result before adding complexity. Simulation comparison does not replace physical testing.

Step-by-step sessions: hours 1-40

Block 1: model and check cases

1-2: review COM and mass weighting, and collect model parameters in SI. 3-4: study linear and angular momentum, force and moment arm. 5-6: derive A for two contacts by hand and check cross-product signs. 7-8: construct a force-and-moment request from bounded body error. 9-10: compare quasi-static and dynamic assumptions, and prepare four and three supports.

11-20 - first practical block: 2 hours for centre of mass and parameters; 2 for the contact matrix; 2 for analytical force and moment examples; 2 for the w_des generator; 2 for comparison with simulation and description of assumptions. Explain the physical meaning of every row of A. If Σf balances weight but the body accelerates downward, check how force is applied and how gravity is modelled before tuning gains.

Save a numerical example: mass, coordinates of four feet, COM, specified forces and expected total force and moment. Label these values as parameters of a particular simulation. Use the example for repeated checks after changing leg order or coordinate frames. Verify its correct answer by hand.

Block 2: QP and computation

21-22: quadratic objective and linear constraints; convert a simple error function into the solver's form. 23-24: friction cone and conservative pyramid. 25-26: a soft force-and-moment error penalty and incompatible hard constraints. 27-28: scaling, regularisation and starting from the previous solution. 29-30: OSQP statuses, independent result checking and the computation-time limit.

31-40 - second practical block: 2 hours for static QP; 2 for three supports and friction; 2 for an unreachable request and independent checking; 2 for cold start and starting from the previous solution; 2 for timing and artificial faults. For a failure, save the forces and original formulation, including active constraints and solver parameters.

By hour 40, the allocator operates on recordings and returns explainable solutions or rejection. Do not connect it to physical motors yet. Closing the loop in simulation adds actuator dynamics, update period and the combined command as new error sources; the next block is specifically allocated to studying them.

Step-by-step sessions and tests: hours 41-75

Integration blocks

41-42: Jacobian and virtual power; 43-44: contact-force signs and dynamics; 45-46: torque, transmission, current and continuous limits; 47-48: impedance, saturation and priorities; 49-50: whole-body control and independent mode switching. Practice 51-60: 2 hours for $J^T f$; 2 for the complete command; 2 for actuator constraints; 2 for closed-loop standing; 2 for contact changes and fallback mode.

61-62: finite horizon and a small MPC example; 63-64: NMPC formulation and slope comparison; hour 65: operating limits and final criteria. Practice 66-75: 2 hours for one-dimensional MPC; 2 for comparing the baseline and QP on a slope; 2 for identical disturbances; 2 for all fault scenarios; 2 for reporting and reproduction. Total: 75 hours, including necessary library installation.

Eight required scenarios

1. Symmetric stance. Equal admissible loads balance weight and produce near-zero moment. Check the force sum independently. **2. A lost support.** Disable a contact after the solver starts. Reject the old result based on contact generation, zero the lost support's force and record the new mode.

3. Unreachable lateral command. Request a force outside the friction cone. Compare solved with high tracking error against infeasible under hard requirements; these are different outcomes. **4. Insufficient actuator torque.** Lower one leg's limit. Check the sum of contact, gravitational and impedance contributions: the complete command must respect the specified limit.

5. A late result. Insert a delay longer than permitted. A new step is prohibited, the mode switch uses the verified policy, and a late answer is not accepted as fresh. **6. Invalid inputs.** NaN, negative mass, an invalid normal and mixed-up units must produce a specific validation error, not an arbitrary command.

7. Low friction on a slope. Actual μ is lower than the model assumes. Measure slipping and body error even with solved status; contact checking must detect the violated assumptions. **8. Return from fallback mode.** Alternate valid and invalid answers near the threshold. Hysteresis must prevent frequent switching while preserving immediate mandatory rejection. Check command continuity and absence of spontaneous gait restart.

What it means to complete the module

Contents of the final package

In qp_stance_v1, save coordinate-frame and sign conventions, model parameters, QP formulation, force-and-moment request generation, actuator adapter, independent validation, classical fallback mode, scenarios and source measurements. The README must include commands for analytical checks, replay, simulation comparison and timing. Attach at least one analysis of a rejected solution with all constraints and the physical reason.

Criterion 1: force and moment balance and units are checked by hand and in code. Criterion 2: accepted QP solutions obey contact constraints and limits on the complete joint command within the specified numerical tolerance. Criterion 3: rejection responses are defined for error statuses, incompatible constraints, iteration limits, timeout and NaN. Criterion 4: fallback mode is tested on current supports and does not use force from an absent leg.

Criterion 5: the timing distribution is measured; CPU, solver version, example count, start methods and deadline misses are reported. Criterion 6: the baseline and QP are compared on identical slopes and disturbances, including failures and saturation. Criterion 7: for whole-body control, MPC and NMPC, implemented work is distinguished from theoretical study. This protocol does not test dynamic running.

Questions without prompts

1) Why must force and moment scales be reconciled in a shared norm? 2) How is the moment of a foot force about the COM calculated? 3) Why can ordinary unilateral contact not pull the foot towards the ground? 4) How does the inner friction diamond differ from the outer square? 5) Why does solved not confirm exact achievement of w_des? 6) When is an initial guess from the previous solution physically invalid?

7) How can virtual power check the sign of $J^T f$? 8) Why are gravitational and impedance contributions considered together? 9) How does winding current differ from supply current? 10) Why does p99 not guarantee an upper time bound? 11) Why must the previous force not be retained after contact loss? 12) What does MPC add to the current support QP? Give an example from your testbed for every answer.

If the time budget is exhausted

If time is short, remove transfer to C++, extensive parameter tuning, acados installation and full inverse dynamics. Retain a working QP, simple model, explicit constraints, fault checks and classical fallback mode. If the QP gait is unstable, restrict the result to standing and slow contact changes, stating this in the report. Hand M21 the unchanged working mode, bounded-correction interface and independent observer; baseline checks must be complete before learning begins.

Assigned reading: forces, constraints and QP

Read the specified extracts before the corresponding tasks. Core reading explains principles; reference reading assists implementation. Sources may be in English; prepare your explanations and report in English. Reading is included in the 75 hours.

Core reading

1. Stephen Boyd, Lieven Vandenberghe, Convex Optimization, 2004. §4.4 Quadratic optimization problems; §5.5.1 and §5.5.3 (stopping criteria; KKT optimality conditions). For Tasks 4-7: write the variables, objective and constraints of your QP; understand feasibility and numerical accuracy.

2. Kevin Lynch, Frank Park, Modern Robotics. §5.2 Statics of Open Chains - derivation through virtual power for Task 8. §11.5 Force Control; §11.6 Hybrid Motion-Force Control - selected reading for Tasks 9-11: distinguish motion control, force control and contact constraints.

Notes for the short MPC experiment

3. Russ Tedrake, Underactuated Robotics. Trajectory Optimization: Direct transcription; Model-Predictive Control; Receding-horizon MPC. For Task 12: a constrained one-dimensional COM model, finite horizon, execution of the first action and replanning. Do not implement full nonlinear body dynamics.

Three sections of solver documentation

OSQP: The solver - Problem statement, convergence and infeasibility for formulating Tasks 4-5. Map every matrix to the testbed's physical quantities by hand.

OSQP: Python interface - setup, solve, update, warm start for Tasks 5-7. Status values and errors - table of statuses and rejection reasons; additionally check physical constraints and result age.

What to retain in your notes

Keep the standard QP formulation, a diagram of admissible forces, the $\tau = J^T f$ check with consistent signs and the rule for accepting a result. Study the structure of Pinocchio, Crocoddyl and acados later; installation is unnecessary for the required force allocation and small MPC example.

Fundamental concepts in M20

Centre of mass (COM). The mean position of a system's mass: $c = \Sigma m_i r_i / \Sigma m_i$; it depends on link poses and mass distribution.

Wrench. Total force and moment about a specified point; components are expressed in N and N·m.

Momentum. A system's linear momentum $p = m \cdot v_{COM}$; its change is determined by the sum of external forces. Angular momentum is described separately.

Normal force. The reaction component along the surface normal; for ordinary unilateral contact, it points away from the surface and is nonnegative.

Friction cone. Admissible contact forces in the Coulomb model: tangential-force norm does not exceed μ times normal force.

Friction pyramid. A polyhedral approximation of a cone using linear constraints; a conservative approximation places its boundaries inside the original cone.

Quadratic programme (QP). Optimisation of a quadratic objective subject to linear equalities and inequalities; here, variables describe stance-leg forces.

Convexity. A property of the objective and feasible set under which a local minimum is global; uniqueness requires additional conditions.

Feasibility. The existence of variable values satisfying every constraint in the formulation; mathematical feasibility does not establish physical-model correctness.

Objective function. The criterion for choosing admissible forces; it usually combines desired force-and-moment error with a penalty on command magnitude.

QP regularisation. An additional penalty on forces or other variables that reduces ambiguity; its magnitude affects the selected solution.

Scaling. Bringing quantities to comparable numerical scales; physical constraints are checked after converting results back to original units.

Residual. A numerical measure of equation or optimality-condition violation; a small value does not prove that axes, parameters and timing are correct.

Incompatible constraints (infeasible). The problem has no solution satisfying all constraints; this differs from a computation deadline miss or input error.

Warm start. Reusing the previous approximation; after contact changes, check component mappings and constraints.

Deadline. The time limit for a control result to remain applicable; beyond it, even a correct answer may rely on stale state.

The mapping $\tau = J^T f$. The relationship between force and joint torques through equality of virtual power, with consistent coordinates, signs and force definition.

Impedance. A specified dynamic relationship between motion error and force; actuator capability, delay and loop discretisation limit achievable behaviour.

MPC. Control through repeated finite-horizon optimisation: after executing the first action, measure state and solve the problem again.

Whole-body control. Coordinated control of a mechanism accounting for dynamics, contacts and actuators; force allocation is only one part.

Equipment and software for M20

Use existing equipment

Use the MacBook Pro M4, the M19 crawl_estimator_v1 testbed, NumPy, SciPy and the existing simulator. ROS runs in Ubuntu 24.04 ARM64 in UTM with Jazzy and Gazebo Harmonic; QP analysis may run directly in macOS ARM. Before integration, preserve classical standing and pause modes and comparison scenarios.

No mandatory purchases

The force allocator requires no new actuators, force sensors, Jetson or NVIDIA GPU. Use previous measurements for physical-actuator limits; explicitly label illustrative simulation parameters. The full leg set is purchased at the physical M22 stage, not for solving the QP.

Install now: one solver

Add osqp to a separate analysis environment or an explicitly designated Python environment for the testbed. Reuse NumPy and scipy.sparse. The official OSQP Python installation and setup/solve/update interface provide the minimum route. Record Python, versions, architecture and solver settings; check ARM64 build availability before choosing the environment.

On macOS ARM, perform calculations on recordings; before closing the ROS loop, test the same problem separately in Ubuntu ARM64. Install Python packages in a compatible environment. If a prebuilt wheel is unavailable, use the documented source build or a verified environment with measured communication latency.

Installation and result checks

Solve a small convex QP with a known answer, then a symmetric stance: four vertical forces balance weight with consistent signs. Check OSQP statuses for feasible and deliberately contradictory formulations. An independent validator checks constraints, finite values, freshness and contact generation.

In a solve/update series, measure cold and warmed-up starts, long delays and deadline misses. On contact loss, exclude that leg's old force. Timeout and infeasible statuses must activate verified stand/pause modes. Check equality of virtual power for $\tau = J^T f$.

Optional, only for a specific need

Pinocchio is needed only if its dynamics algorithms are used; Crocoddyl and acados are unnecessary for the required QP and small MPC example. Reuse MuJoCo only if already selected in M19. An accelerator may be purchased after measurements and the G3 decision on 27.11-10.12.2028; this future checkpoint is not treated as completed here.

Preparation and short functionality checks are included within the original 75 hours instead of reinstalling the environment. For M21, retain the classical baseline unchanged with its configuration and fault-test results.

Motion fundamentals: a floating body

Full dynamics model and the implementation limits of whole-body control

Configuration, velocity and available actuators

For rigid links with 12 revolute joints, q=(p_B,Q_WB,q_j): body position in W, a unit quaternion for rotation B→W and joint angles. Here n_q=3+4+12=**19**, but v=(v_B,ω_B,$qdot_j$) contains 3+3+12=**18** components. Express v_B and ω_B in W, in m/s and rad/s. The quaternion has a norm constraint, so qdot=N(q)v, where N has size 19×18; do not substitute the 19-component qdot for v.

M(q)·vdot + h(q,v) = S^Tτ + J_c^Tf_ext. M is the 18×18 inertia matrix; h includes velocity-dependent and gravitational terms. S=[$0_{12\times 6}$ I_{12}] selects the 12 actuated velocities: S^Tτ has six zeros for the base. The first six equations cannot be satisfied by separate “body motors”. Generalised forces have units N for translational and N·m for angular components.

Contact links motion and forces

For k stationary, non-slipping point supports, J_c has size 3k×18 and maps v to foot velocities in W. While this mode persists, **J_c v=0, J_c vdot+Jdot_c v=0**. f_ext denotes environment forces on the robot, in N; unilateral-contact and friction constraints also apply. During swing, the corresponding force is zero; during slipping, the zero-foot-velocity condition cannot be retained.

In M20, the wrench is ordered [force; moment]: w_C=[Σf_i; Σ((p_i−c)×f_i)] about COM c, all in W. A point contact transmits no moment of its own; force acting at a moment arm produces the moment about COM. A contact patch could also transmit its own moment, but the current QP uses point feet.

A small example: check the sign before optimisation

Let c=(0,0,0.40) m, p=(0.20,0.15,0) m and f=(0,0,30) N. Then r=p−c=(0.20,0.15,−0.40) m and **r×f=(4.5,−6,0) N·m**. For a symmetric stance with a mass of 10 kg, four vertical forces of 24.525 N balance the 98.1 N weight and produce zero total moment. This checks equilibrium, not compliance with joint limits.

In statics, the actuated rows give **τ=h_j−(J_c^Tf_ext)$_j$**. If h_j=3 N·m in one joint and the external force contributes +5 N·m, the required torque is −2 N·m. Therefore, +J^Tf_ext must not automatically be sent to motors. Virtual power f_{ext}^T(J_c v)=(J_c^Tf_ext)^{T}v checks the consistency of the mapping.

Connect the model to the existing experiment

In Tasks 2 and 8, create shapes.md for q,v,M,S,J_c. Check that S^Tτ does not act directly on the base. In the analytical example, save force_balance.csv with total force, moment and the static sign of τ. Disable a support: the old forces must fail the new constraints. Do not send computed torque to a position-controlled actuator; the adapter must match its available interface.

Hours: 1 hour from 43-44 on dynamics and 1 hour from 51-60 for the force-mapping test. The model is required; a new inverse-dynamics solver is unnecessary. Existing QP, fallback mode, deadlines and 75 hours are retained. Read: Tedrake, Multi-Body Dynamics, “The Manipulator Equations” and “Bilateral Position Constraints”; Centroidal dynamics / Generalization to multibody.